

Theory / Essay Prep — Test 2

STQD6114 | Image & Audio Data Analysis

PART 1 — IMAGE DATA ANALYSIS

Summary (Quick Reference)

What is a Digital Image?

- A 2D function $f(\mathbf{x}, \mathbf{y})$ where f = light intensity at position (x, y)
- A **pixel** (picture element) is each discrete point with a location and value
- If x , y , and f are finite and discrete → **digital image**

Image Types

Type	Range	Channels
Binary	{0, 1}	1
Grayscale	{0, ..., 255}	1
Color (RGB)	{0, ..., 255} each	3 (R, G, B)

Phases of Image Processing

1. **Image Acquisition** — camera captures & digitizes (illumination → optical lens → sensor)
2. **Image Enhancement** — improve image for specific use (denoising, deblurring, sharpening)
3. **Image Restoration** — recover degraded image using mathematical/probabilistic models
4. **Morphological Processing** — extract shape components (erosion, dilation)
5. **Image Segmentation** — separate objects from background
6. **Object Recognition** — assign label to object based on description
7. **Representation & Description** — convert to computer-processable form; extract features
8. **Image Compression** — reduce storage/bandwidth (JPEG ~16:1, DVD ~48:1)
9. **Color Processing** — handle coloured image channels

Processing Levels

- **Low level:** input=image, output=image (noise reduction, sharpening)
- **Mid level:** input=image, output=attributes (edges, segmentation)
- **High level:** input=attributes, output=decisions (face recognition, autonomous navigation)

Image Acquisition

- **Sampling** = discretization of spatial coordinates (x, y) → controls resolution (DPI)
- **Quantization** = discretization of amplitude/intensity values → controls gray levels
- More sampling (higher DPI) = sharper image

- More quantization bits (8-bit = 256 levels, 4-bit = 16 levels) = smoother gradients
- Low freq areas more sensitive to quantization artifacts

Histogram & Histogram Equalization

- **Histogram** = frequency of occurrence of each gray level; global image description
- **PDF** = frequency / total pixels
- **CDF** = cumulative sum of PDF values
- **New gray level** = $\text{round}(\text{CDF} \times (L-1))$ where L = number of gray levels ($L-1 = 7$ for 8 gray levels)
- Goal: flatten the histogram → improves contrast

Morphological Operations

Operation	Effect	Use Case
Erosion	Shrinks pixels; removes boundary pixels	Split joined objects; strip extrusions
Dilation	Expands pixels; adds boundary pixels	Repair breaks; fill intrusions

Image — 10 Essay/Theory Questions

Q1 — What is a digital image? Explain the mathematical representation.

Model Answer:

A digital image is a projection of a 3D scene onto a 2D plane. Mathematically, it is represented as a two-variable function $f(x, y)$, where x and y are spatial coordinates and f gives the light intensity (gray level) at that point.

When the values of x , y , and f are all finite and discrete, the image is called a **digital image**. It is composed of a finite number of elements called **pixels** (picture elements), each with a specific location and intensity value.

For a grayscale image: $g(x, y) \in C$ where typically $C = \{0, 1, \dots, 255\}$ (8-bit).

For a colour image, three channels are used: $gR(x, y)$, $gG(x, y)$, $gB(x, y)$.

Q2 — Describe the three levels of digital image processing with examples.

Model Answer:

1. Low-level processing

- Both input and output are images.
- Performs primitive operations such as noise reduction, image sharpening, and enhancement.
- Example: Removing salt-and-pepper noise from a photo.

2. Mid-level processing

- Input is an image; output is attributes or features extracted from the image.
- Includes segmentation and classification of individual objects.

- Example: Detecting edges of a car in an image.

3. High-level processing

- Input is attributes/features; output is decisions or understanding.
 - Involves making sense of recognised objects.
 - Example: Automatic character recognition (OCR), autonomous vehicle navigation, face recognition.
-

Q3 — Explain image sampling and quantization. What is the effect of reducing each?

Model Answer:

Image Sampling refers to the discretization of spatial coordinates (x and y). When a continuous image is projected onto a sensor, it is divided into discrete picture elements (pixels). The number of samples taken per unit distance is measured in **DPI (dots per inch)**.

- **Effect of reducing sampling:** Lower DPI → fewer pixels → image appears blurry or pixelated. High-quality images use high DPI (e.g., 1250 dpi vs 72 dpi shows a dramatic difference in sharpness).

Image Quantization refers to the discretization of intensity (amplitude) values. It determines how many gray levels are available.

- 8 bits = 256 gray levels; 4 bits = 16 gray levels; 1 bit = 2 levels (binary).
- **Effect of reducing quantization:** Fewer gray levels → visible banding/contouring in smooth regions. Low-frequency areas (smooth regions) are more sensitive to quantization artifacts.

Both must be balanced: higher sampling and quantization improve quality but increase file size.

Q4 — List and describe at least five applications of digital image processing.

Model Answer:

1. **Medical and Biomedical** — Surgical assistance, sensor fusion, vision-based diagnosis (e.g., detecting tumours in MRI scans).
 2. **Automotive Driver Assistance** — Lane departure warning, adaptive cruise control, obstacle warning (e.g., MobilEye system).
 3. **Security and Safety** — Biometric verification (face, iris recognition), surveillance of fences and swimming pools.
 4. **Industrial Automation** — Vision-guided robotics, quality inspection systems on production lines.
 5. **Astronomy** — Enhancement of astronomical images, chemical/spectral analysis of planets.
 6. **Digital Photography** — Image enhancement, compression, colour manipulation, editing in digital cameras.
 7. **Military** — Tracking and localizing targets, missile guidance, detection systems.
 8. **Traffic Monitoring** — Adaptive traffic lights, traffic flow monitoring using cameras.
-

Q5 — Explain the phases of image acquisition. What are the three principal sensor arrangements?

Model Answer:

Image acquisition is the first phase of image processing, where the real-world scene is captured and converted into a digital image. It involves three main components:

1. **Illumination** — Light source that illuminates the scene.
2. **Optical system (lens system)** — Focuses the light onto the sensor.
3. **Sensor system** — Converts optical energy into electrical/digital signals.

The relationship is: $f(x,y) = i(x,y) \times r(x,y)$ where i = illumination component, r = reflectance component.

Three principal sensor arrangements:

Sensor	Description
Single sensor	One sensor element; slow but high precision
Line sensor	A row of sensors; scans line by line
Array sensor	2D grid of sensors (most modern cameras); captures the full image at once

Q6 — What is Histogram Equalization? Explain the steps with a worked example.

Model Answer:

Histogram Equalization is an image enhancement technique that spreads out the gray level distribution to improve contrast. It works by normalizing the histogram to a flatter profile.

Steps:

1. Count the **frequency** of each gray level.
2. Compute **PDF** = frequency / total pixels.
3. Compute **CDF** = cumulative sum of PDFs.
4. Compute **New gray level** = round($CDF \times (L-1)$), where L = number of gray levels (use $L-1 = 7$ for 8 gray-level image, 0–7).
5. Map original pixels to new gray levels.

Worked Example (3-bit image, $L=8$, $L-1=7$, total pixels = 16):

GL	Freq	PDF	CDF	CDF × 7	New GL
0	2	0.125	0.125	0.875	1
1	3	0.188	0.313	2.188	2
2	4	0.250	0.563	3.938	4
3	3	0.188	0.750	5.250	5
4	3	0.188	0.938	6.563	7
5	1	0.063	1.000	7.000	7

Each original pixel is then replaced by its mapped New GL. This spreads dark pixels into lighter bins, improving contrast.

Key formula: $\text{New GL} = \text{round}(\text{CDF}(k) \times (L - 1))$

Q7 — Differentiate between image enhancement, image restoration, and image inpainting.

Model Answer:

Technique	Definition	Method
Enhancement	Improves an image to make it more suitable for a specific application — subjective	Applied without knowing how degradation occurred (e.g., denoising, sharpening, histogram equalization)
Restoration	Recovers an image that has been degraded — objective	Uses mathematical or probabilistic models of the degradation process (e.g., deblurring using inverse filter)
Inpainting	Fills in missing or damaged regions of an image	Uses surrounding pixels to reconstruct missing content (e.g., restoring old photographs, removing watermarks)

Example: A blurry satellite image → **restoration** (deblurring). An overexposed photo → **enhancement** (histogram adjustment). A torn historical painting → **inpainting**.

Q8 — Explain morphological image processing. Compare erosion and dilation.

Model Answer:

Morphological image processing is a set of operations that process images based on **shape**. A **structuring element** (a small binary matrix) is applied to the input image; each output pixel value is determined by comparing the input pixel to its neighbourhood according to the structuring element.

Erosion:

- **Shrinks** the objects in an image by removing pixels on object boundaries.
- Properties:
 - Can split apart joined objects (e.g., two touching blobs become separated).
 - Strips away protrusions/extrusions from object edges.
- Use case: Separating touching cells in a microscopy image.

Dilation:

- **Expands** the objects in an image by adding pixels to object boundaries.
- Properties:
 - Can repair breaks in lines or contours.
 - Fills small holes/intrusions within objects.
- Use case: Reconnecting a broken line in a handwritten digit.

Together (erosion then dilation = **opening**; dilation then erosion = **closing**) they are used to clean up binary images.

Q9 — What is image compression? Give real-world examples with compression ratios.

Model Answer:

Image compression is the technique of reducing the storage space required to save an image or the bandwidth required to transmit it. It works by removing redundant or perceptually irrelevant data.

Types:

- **Lossless:** No information is lost (e.g., PNG, FLAC for audio).
- **Lossy:** Some information is discarded in exchange for much higher compression (e.g., JPEG).

Real-world examples from notes:

Scenario	Uncompressed	Compressed	Ratio
600×800 colour image (24-bit)	1.44 MB	89 KB (JPEG)	~16:1
Movie (720×480, 30fps, 24-bit)	~243 Mbits/sec	~5 Mbits/sec (DVD)	~48:1

Why compression matters: Without it, even a 2-second HD clip would consume hundreds of megabytes, making streaming and storage impractical.

Q10 — Explain adaptive histogram equalization and the various uses of histograms in image analysis.

Model Answer:

Standard histogram equalization operates on the entire image globally. However, this can over-enhance certain regions while ignoring local variations.

Adaptive Histogram Equalization (AHE) computes histograms for local sub-regions of the image independently and applies equalization to each region. This gives much better local contrast enhancement.

Use cases for adaptive histogram:

- Pattern detection
- Adaptive enhancement (e.g., brightening only dark regions)
- Adaptive thresholding (setting different thresholds for different regions)
- Object tracking

General uses of histograms in image processing:

1. **Digitizing parameters** — choosing sampling/quantization settings
2. **Measuring image properties:** average, variance, entropy, contrast, area for a given gray-level range
3. **Threshold selection** — finding the boundary between object and background
4. **Image distance** — comparing two images by comparing their histograms
5. **Image enhancement** — histogram equalization, stretching, and matching

PART 2 — AUDIO DATA ANALYSIS

Summary (Quick Reference)

Audio Signal Basics

- Audio signals represent sound in **digital** or **analog** form
- Human hearing range: **20 Hz to 20,000 Hz**
- Most sensitive range: **500 Hz to 5000 Hz**
- Infrasound (< 20 Hz) still affects the ear even if inaudible; 7 Hz is especially dangerous (resonates with body organs)
- **Pitch** corresponds to **frequency** (wavelength)
- **Loudness/Volume** corresponds to **amplitude**
- Shorter wavelength → higher frequency → higher pitch

Signal Types

Signal	Time	Amplitude
Analog	Continuous	Continuous
Digital	Discrete	Discrete

Digital Signal Processing (DSP)

- DSP: representation of signals as sequences of numbers, and processing those sequences
- Signal = a function that conveys information about a physical system's state
- **Advantages of DSP:** flexibility, error detection/correction, easier data storage, low cost
- **Disadvantages of DSP:** higher power consumption, steeper learning curve

Key DSP Concepts

1. **Fourier Transforms** — decompose signals into frequency components
2. **Noise** — any unwanted signal; must be understood and removed/managed
3. **Filters** — remove specific frequency portions from a signal

Fourier Transform & STFT

- **Fourier Transform (FT):** reversible mathematical transform (Joseph Fourier, early 1800s)
- Breaks a time-series into a **sum of sine and cosine functions**
- **DFT (Discrete Fourier Transform):** applied to discrete time-domain signals
- Switches signal from **time domain** → **frequency domain**

Term	Meaning
FFT	Fast Fourier Transform — efficient algorithm to compute DFT
Window size	Controls length of the FT; larger = more freq resolution, less time accuracy

Term	Meaning
STFT	Short-Time Fourier Transform — slides a window along the signal, computing DFT at each step
Spectrogram	2D plot of STFT results: X=time, Y=frequency, colour=amplitude

Audio File Formats in R

Format	Type	Description
.wav	Uncompressed	Full information; large file
.mp3	Lossy compressed	Information reduced; time/amplitude/frequency can be impaired
.flac	Lossless compressed	Full information in a smaller file

Key R Functions (tuneR + seewave)

```

readWave()      - load .wav
readMP3()       - load .mp3
fft(z@left)     - FFT on left channel
timer()         - detect sound vs silence periods
dynspec()       - dynamic spectrogram (STFT) with oscillogram
spectro()       - static spectrogram
meanspec()      - average spectrum over whole duration

```

Audio — 10 Essay/Theory Questions

Q1 — What is an audio signal? Describe its key properties with reference to human hearing.

Model Answer:

An **audio signal** is a representation of sound in the form of digital or analog signals. Sound is a pressure wave that travels through a medium (usually air), and audio signals capture the variation in that pressure over time.

Key properties:

- **Frequency (Hz):** Determines the pitch of the sound. Human hearing detects frequencies between **20 Hz and 20,000 Hz**.
- **Amplitude:** Determines the loudness/volume of the sound.
- **Wavelength:** Inversely related to frequency — shorter wavelength = higher frequency = higher pitch.

Human hearing specifics:

- Most sensitive range: **500 Hz to 5000 Hz** (where most speech information lies).
- Sounds below 20 Hz (infrasound) are inaudible but still affect the body.

- Particularly dangerous: **7 Hz infrasound**, which is close to the resonant frequencies of human organs and may disturb heart or brain activity.

Relationship:

- Pitch depends on **frequency**
- Loudness depends on **amplitude**

Q2 — Differentiate between analog and digital signals. Why is digitization important for audio processing?

Model Answer:

Feature	Analog Signal	Digital Signal
Time	Continuous	Discrete
Amplitude	Continuous	Discrete
Storage	Difficult, degrades over time	Easy, no degradation
Processing	Analog circuits (passive/active elements)	Computers, DSP chips, FPGAs, ASICs
Error correction	None	Possible

Analog signals are continuous in both time and amplitude. They closely follow the physical waveform but are susceptible to noise and degradation during transmission.

Digital signals are sampled at discrete time intervals and quantized to discrete amplitude levels. They encode sound as a binary sequence (0s and 1s).

Why digitization matters:

- Digital files can be stored, copied, and transmitted without quality loss.
- Enables powerful mathematical operations (FFT, filtering, compression).
- Easier to process with computers.
- Error detection and correction is possible.
- Lower cost at scale.

Q3 — What is Digital Signal Processing (DSP)? List its applications and the advantages/disadvantages.

Model Answer:

Digital Signal Processing (DSP) is concerned with the representation of signals as sequences of numbers or symbols, and the processing of these sequences to extract meaningful information or modify the signal.

A **signal** is a function that conveys information about the state or behaviour of a physical system. Mathematically, signals are represented as functions of one or more independent variables.

DSP involves: analysing, modifying, and synthesizing signals to extract meaning.

Applications:

1. Speech and audio processing (noise reduction, speech recognition)
2. Image and video processing (compression, filtering)
3. Military and telecommunications (radar, encrypted communications)
4. Healthcare and biomedical (ECG/EEG analysis, ultrasound)
5. Consumer electronics (smartphones, hearing aids)

Advantages:

- Flexibility — algorithms can be changed via software
- Error detection and correction features
- Easier data storage
- Low cost

Disadvantages:

- Higher power consumption than analog systems
- Higher learning curve required for implementation and operation

Q4 — Explain the Fourier Transform and the Discrete Fourier Transform (DFT). Why are they important in audio analysis?

Model Answer:

Fourier Transform (FT) is a reversible mathematical transform developed by **Joseph Fourier in the early 1800s**. It decomposes a time-domain signal into a sum of sine and cosine functions of different frequencies, revealing the **frequency content** of the signal.

Any signal can be broken down into a series of sine and cosine curves.

The FT gives us the **frequency spectrum** of a signal — showing which frequencies are present and at what strength (amplitude).

Discrete Fourier Transform (DFT):

- A specific form of Fourier Transform applied to **discrete** (sampled) time signals.
- Takes a sequence of N time-domain samples and produces N complex frequency-domain coefficients.
- The **FFT (Fast Fourier Transform)** is an efficient algorithm for computing the DFT.
- Switching from time domain → frequency domain using amplitude and frequency of the signal.

Why important in audio:

- A raw audio waveform shows how amplitude changes over time, but not what frequencies are present.
- The DFT/FFT reveals the **dominant frequencies** in the sound (e.g., the pitch of a note).
- Enables frequency-based filtering, noise removal, pitch detection, and music analysis.

In R: `fft(z@left)` computes the FFT on the left channel of a Wave object.

Q5 — What is the Short-Time Fourier Transform (STFT)? How does it differ from the regular Fourier Transform, and what is a spectrogram?

Model Answer:

The limitation of regular FT: Applying a Fourier Transform to the whole audio signal shows the overall frequency content but loses all **temporal information** — you cannot tell *when* a frequency occurs. This is insufficient for real-world audio where frequencies change over time (e.g., speech, music).

Short-Time Fourier Transform (STFT):

- A **sliding window** is moved along the signal.
- A DFT is computed at each position of the window.
- This produces frequency information at each moment in time.
- Allows the signal to be represented in the **frequency spectrum while maintaining time information**.

Window size trade-off:

- **Larger window** → higher frequency resolution, but lower time precision (more signal is captured per transform).
- **Smaller window** → better time precision, but lower frequency resolution.

Spectrogram:

- A 2D visual plot of STFT results.
- **X-axis** = time, **Y-axis** = frequency, **colour/intensity** = amplitude.
- Displays three variables in a 2D plot (avoids a hard-to-read 3D plot).
- Widely used in bioacoustics, acoustics, and ecoacoustics.

In R: `spectro(z)` produces a static spectrogram; `dynspec(z, wl=1024, osc=T)` produces a dynamic one (STFT with oscillogram).

Q6 — What are the three types of audio file formats used in R? Compare them.

Model Answer:

Within R, digitized sound can be stored in three categories of files:

Format	Type	Quality	File Size
.wav	Uncompressed	Full — all information retained	Large
.mp3	Lossy compressed	Reduced — time, amplitude, and frequency can be impaired	Small
.flac	Lossless compressed	Full — all information retained in reduced size	Medium

All formats store sound encoded as a succession of binary (0 and 1) values.

Detail:

- **WAV:** The gold standard for analysis — no data loss. Preferred for scientific audio analysis in R (`readWave()`).

- **MP3:** Common for music distribution. Uses perceptual encoding to discard frequencies humans are less sensitive to. Suitable for listening but not ideal for precise analysis.
- **FLAC:** Best of both worlds for storage — same quality as WAV in a smaller file. Used by audiophiles and archivists.

For analysis in R: Always prefer WAV where possible. If given MP3, load with `readMP3()`. If given MP4 or other formats, convert first using the `av` package.

Q7 — Explain the concept of the Fourier Transform window size. What happens if the window is too large or too small?

Model Answer:

When applying the Fourier Transform to audio signals, the analysis is typically done on segments (windows) of the signal rather than the entire signal at once.

Window size (controlled by parameter `wl` in `dynspec()`) determines how many samples are included in each DFT computation.

Effect of window size:

Window Size	Frequency Resolution	Time Resolution	Trade-off
Large	High — can distinguish close frequencies	Low — less precise about <i>when</i>	Good for slow-changing signals
Small	Low — close frequencies merge	High — precise timing	Good for fast-changing signals

Why this matters:

- In music analysis, a large window helps identify the exact pitch of a sustained note.
- In speech analysis, a smaller window captures the rapid changes between phonemes.
- Computing FT on a whole sound or single segment alone may not give enough information — STFT (sliding window) is the practical solution.

In R: `dynspec(z, wl=1024, osc=T)` uses a window length of 1024 samples.

Q8 — Describe how the `timer()` function works in audio analysis. What does it detect and what are its parameters?

Model Answer:

The `timer()` function from the `seewave` package detects and visualises periods of **sound vs silence** within an audio signal. It is useful for identifying when sounds occur, how long they last, and the silent gaps between them.

How it works:

1. Computes the energy envelope of the audio signal over time.
2. Compares the envelope against a **threshold** (expressed as a percentage of the maximum energy).
3. Time segments where energy exceeds the threshold are labelled as **sound**; segments below are labelled **silence**.
4. The result is plotted with coloured regions (blue = sound, white = silence).

Parameters:

```
timer(z, f=22050, threshold=20, msMOOTH=c(50,0))
```

Parameter	Meaning
<code>z</code>	The Wave object to analyse
<code>f</code>	Sampling rate in Hz — must match <code>z@samp.rate</code>
<code>threshold</code>	% of max energy below which = silence (lower = more sensitive)
<code>msMOOTH</code>	Smoothing window — reduces noise in detection (larger = smoother)

Important: `f` must match the audio's actual sampling rate. Use `z@samp.rate` rather than hardcoding a value.

Limitation: `timer()` only shows WHEN sound occurs — not the frequency or amplitude. Use FFT for frequency analysis.

Q9 — Explain what `meanspec()` and `spectro()` show, and when each is useful.

Model Answer:

Both functions are from the **seewave** package and provide different views of the frequency content of audio.

`spectro(z)` — Static Spectrogram:

- Plots the STFT (Short-Time Fourier Transform) of the signal.
- X-axis = time, Y-axis = frequency, colour/intensity = amplitude.
- Shows how the frequency content **changes over time**.
- Useful for: speech analysis, bird call analysis, detecting when specific frequencies appear.
- Equivalent to a snapshot of all DFTs along the full duration.

`meanspec(z)` — Mean Spectrum:

- Computes and plots the **average frequency spectrum** across the entire signal duration.
- X-axis = frequency, Y-axis = average amplitude at that frequency.
- Does NOT show time variation — it collapses everything into a single curve.
- Useful for: comparing the overall frequency profile of two signals, identifying dominant frequencies on average.

Comparison:

Feature	spectro()	meanspec()
Shows time variation?	Yes	No
Shows frequency content?	Yes	Yes
Best for	Temporal analysis	Overall frequency profile

In R:

```
spectro(z)           # static spectrogram
meanspec(z)          # average spectrum
dynspec(z, wl=1024, osc=T) # dynamic spectrogram (interactive STFT)
```

Q10 — Why is the FFT output mirrored, and how should you handle it when plotting or analysing audio?

Model Answer:**Why the FFT is mirrored:**

The FFT of a real-valued signal (like audio) produces **N complex frequency coefficients** for N input samples. Due to the mathematical property of the DFT applied to real signals, the output is **symmetric (Hermitian symmetric)**:

- The first half of the FFT output (bins 1 to N/2) contains the **positive frequencies**.
- The second half (bins N/2+1 to N) is a **mirror image** (complex conjugate) of the first half.
- No new information is contained in the second half.

Practical consequence:

- Plotting the full FFT shows a double-peaked, symmetric spectrum that can be confusing.
- Only the first half `Zk[1:(length(Zk)/2)]` is needed for analysis.

In `plot.frequency.spectrum()`, the scaling $\times 2$ for all bins except the DC (bin 1) accounts for this — it restores the true amplitude since the mirror half was discarded.

How to handle it:

```
Zk <- fft(z@left)

# Use only first half for analysis
half_n <- length(Zk) / 2
plot.frequency.spectrum(Zk[1:half_n])

# To find peak frequency correctly:
mags <- Mod(Zk[1:half_n])
mags[2:length(mags)] <- 2 * mags[2:length(mags)]
```

```
peak_bin <- which.max(mags)
peak_freq <- (peak_bin - 1) * z@samp.rate / length(Zk) # convert bin to Hz
```

Rule of thumb: always work with `Zk[1:(length(Zk)/2)]` when you want to interpret frequency content — never plot or analyse the full mirrored FFT.

HISTOGRAM EQUALIZATION — STEP BY STEP FORMULA CARD

Step 1: Count frequency of each gray level

Step 2: $PDF(k) = freq(k) / total_pixels$

Step 3: $CDF(k) = PDF(0) + PDF(1) + \dots + PDF(k)$ [cumulative sum]

Step 4: $new_GL(k) = round(CDF(k) \times (L - 1))$ [L-1 = 7 if 8 gray levels (0-7)]

Step 5: Replace each original pixel with its new_GL

L = number of gray levels. For a 3-bit image (0–7): L=8, L–1=7.
For a full 8-bit image (0–255): L=256, L–1=255.

KEY FORMULAS TO REMEMBER

Formula	Description
$f(x,y) = i(x,y) \times r(x,y)$	Image = illumination × reflectance
$PDF(k) = freq(k) / N$	Probability of each gray level
$CDF(k) = \sum PDF(0..k)$	Cumulative distribution
$New\ GL = round(CDF \times (L-1))$	Histogram equalization mapping
$peak_freq = (bin-1) \times samp.rate / N$	Convert FFT bin index to Hz
$bin_limit = target_Hz \times N / samp.rate$	Convert Hz to FFT bin count
$duration = length(z@left) / z@samp.rate$	Audio duration in seconds
$sample_index = time_seconds \times samp.rate$	Convert time to sample position